Propuesta inicial de organización y diseño del proyecto

Comparto esta propuesta para que tengamos una idea común del Sistema de Gestión Académica antes de empezar a programar. Mi intención es que construyamos un proyecto organizado, comprensible y fácil de ampliar durante las actividades 1 y 2. Las recomendaciones de este documento están abiertas a discusión y deben ajustarse a las indicaciones de la profesora.

Qué vamos a construir

En la actividad 1, el sistema representará personas vinculadas a la universidad, programas académicos y asignaturas. Permitirá consultar su información y modificar datos mediante métodos con validaciones. También mostrará las diferencias entre estudiantes, docentes y administrativos mediante herencia, sobreescritura y polimorfismo.

Por ejemplo, podremos representar el programa Ingeniería de Sistemas, la asignatura Programación I y una estudiante que pertenece a ese programa. En esta primera entrega, pertenecer a un programa no significa estar matriculada en un grupo de una asignatura.

En la actividad 2, ampliaremos el mismo proyecto para crear grupos académicos con una asignatura, un docente, estudiantes, capacidad y horario. Incorporaremos matrícula, retiro, búsquedas y excepciones personalizadas. También convertiremos Persona en una clase abstracta.

La actividad 3 corresponde a un proyecto distinto, en un contexto diferente al académico. Esta propuesta se concentra en las actividades 1 y 2.

Requisitos que debemos cumplir

Estos puntos proceden de los documentos de las actividades; no son funcionalidades adicionales que esté proponiendo.

Clase de la actividad 1	Información mínima
Persona	Identificación, nombre y correo electrónico.
ProgramaAcademico	Código, nombre, facultad y número de semestres.
Asignatura	Código, nombre, créditos y programa académico al que pertenece.
Estudiante	Datos comunes de Persona, código estudiantil, programa, semestre y promedio acumulado.
Docente	Datos comunes de Persona, número de empleado, facultad, tipo de contratación y horas semanales.
Administrativo	Datos comunes de Persona, número de empleado, dependencia, cargo y jornada.

Las clases deben utilizar constructores y atributos encapsulados. Debemos demostrar modificaciones válidas y rechazar, como mínimo, un correo vacío, un número de semestres menor o igual a cero y un número de créditos menor o igual a cero.

Estudiante, Docente y Administrativo deben heredar de Persona y sobrescribir mostrar_informacion() y realizar_actividad_principal(). La demostración debe incluir al menos dos estudiantes, dos docentes y dos

administrativos en una sola colección, recorrida con un único ciclo que invoque los mismos métodos sin condicionales por tipo.

La entrega requiere un proyecto ejecutable desde un archivo principal, código organizado y documentado y un README con integrantes, descripción, estructura, instrucciones y explicación de los conceptos aplicados. Cada integrante debe realizar al menos un commit significativo relacionado con el desarrollo de cada actividad. En la actividad 1 debemos declarar el uso de IA en el README.

Para la actividad 2 debemos conservar identificable la primera entrega, crear al menos dos grupos y demostrar operaciones normales y excepcionales. Se requieren cinco situaciones controladas con excepciones propias: matrícula duplicada, grupo lleno, retiro de un estudiante no matriculado, asignatura inexistente y programa inexistente. Las excepciones deben lanzarse donde se detecta el problema y capturarse en el programa principal, permitiendo continuar la ejecución.

La segunda entrega incluye un Documento de Decisiones de Diseño, la declaración del uso de IA en ese documento y el tag Entrega_AC2. Debemos preservar la versión de la primera entrega sin alterar su contenido histórico.

Por qué propongo MVC

Propongo seguir la organización Modelo–Vista–Controlador que la profesora está mostrando en clase. Aunque los documentos de las actividades no nombran MVC expresamente, esta organización nos ayuda a separar responsabilidades y a mantener una estructura coherente con sus ejemplos.

Parte	Responsabilidad en nuestro sistema
Modelo	Representar las entidades académicas, proteger sus datos y aplicar las reglas del negocio.
Vista	Mostrar información, solicitar datos y presentar mensajes mediante la interfaz.
Controlador	Coordinar las acciones de la vista con las operaciones del modelo.
main.py	Preparar e iniciar el sistema y ofrecer la demostración exigida por la actividad.

Por ejemplo, en la actividad 2 la vista podría solicitar los datos para una matrícula. El controlador buscaría los objetos y pediría la operación a GrupoAcademico. El grupo comprobaría duplicados y capacidad. La vista presentaría el resultado. Para cumplir literalmente el requisito de captura en el programa principal, mantendríamos allí la demostración de las excepciones; la interfaz también necesitaría controlar esos errores en sus acciones.

La ventaja es que los botones y las ventanas no contendrían las reglas académicas. Podríamos modificar la presentación sin reescribir la lógica de las clases. Cada integrante también podría localizar con más facilidad dónde corresponde realizar un cambio.

Organización de carpetas que sugiero

La siguiente estructura es una propuesta inicial, no una estructura que ya hayamos creado:

```
sistema-gestion-academica/
+-- main.py
+-- models/
+-- views/
+-- controllers/
+-- docs/
+-- README.md
```

```
+-- requirements.txt
```

En models guardaríamos las clases del dominio; en views, las pantallas; en controllers, la coordinación de acciones; y en docs, la documentación de diseño. requirements.txt registraría las dependencias externas que finalmente utilicemos.

Añadiríamos módulos para las excepciones y las búsquedas cuando sean necesarios en la actividad 2. Prefiero que la estructura crezca con las necesidades del proyecto y que evitemos crear archivos o clases sin una responsabilidad concreta.

Interfaz y almacenamiento

Me gustaría que el sistema tenga una interfaz gráfica cuidada, con formularios claros, tablas legibles y una navegación sencilla. Propongo evaluar PySide6 con Qt Widgets como primera opción. CustomTkinter sería una alternativa si queremos reducir el tiempo dedicado a aprender la herramienta. La elección de la librería sigue pendiente de acuerdo entre todas.

Los documentos no exigen una interfaz gráfica ni una base de datos. Por eso sugiero comenzar con objetos almacenados en listas o diccionarios en memoria. Esto significa que los cambios no se conservarían al cerrar el programa. Si después consideramos necesario guardar registros entre ejecuciones, podemos evaluar SQLite como una ampliación.

Tampoco necesitamos un CRUD completo para cada entidad. Priorizaremos crear, consultar y modificar lo solicitado. En la actividad 2, retirar a una estudiante de un grupo no implica eliminar a esa estudiante del sistema.

Aunque incorporemos una interfaz, propongo conservar una demostración reproducible desde el programa principal para comprobar todos los casos exigidos, incluidos los valores inválidos y el recorrido polimórfico.

Prácticas que recomiendo

Sugiero utilizar nombres claros, métodos con una responsabilidad definida y validaciones dentro de la clase que controla el dato o la operación. Por ejemplo, GrupoAcademico debe controlar sus cupos y la matrícula; esas comprobaciones no deben depender únicamente de un botón de la interfaz.

También propongo reutilizar las validaciones al construir y modificar objetos, siguiendo el criterio que aparece en el ejemplo de la profesora. Así evitamos que un objeto nazca con datos que el propio sistema rechazaría al actualizarlo.

Antes de programar, recomiendo elaborar un diagrama de clases sencillo de la actividad 1 y actualizarlo en la actividad 2. No es un entregable obligatorio para estas dos actividades, pero nos ayudará a revisar atributos, métodos, herencia y relaciones. El diagrama debe reflejar el código que finalmente implementemos.

Cómo sugiero trabajar en equipo

Propongo revisar primero el ejemplo storeProject de la profesora para acordar convenciones de nombres, organización y validación. Después podemos distribuir tareas concretas según las responsabilidades de las clases y de la interfaz.

Cada integrante debe trabajar con su propia cuenta de GitHub y aportar commits con cambios coherentes y mensajes claros. El requisito es al menos un commit significativo por integrante y por actividad; realizar commits frecuentes sería una práctica nuestra para conservar el avance, no una exigencia de registrar cada pequeña edición.

Antes de integrar un aporte, sugiero que otra compañera lo revise y que comprobemos que funciona con el resto del sistema. No considero necesario adoptar una metodología formal como Kanban para este alcance; podemos coordinarnos con una lista breve de tareas y responsables.

Todas debemos poder explicar el proyecto completo, además de la parte que desarrollamos. Si utilizamos IA, debemos registrar para qué la usamos y verificar sus propuestas antes de incorporarlas.

Decisiones pendientes del equipo

- Confirmar la librería de interfaz y el alcance de las pantallas de la primera entrega.
- Acordar el diagrama inicial y las convenciones a partir del ejemplo de la profesora.
- Distribuir las responsabilidades y definir cómo revisaremos e integraremos los aportes.
- Confirmar cómo identificar la entrega de la actividad 1; su documento menciona una etiqueta, pero no indica un nombre específico.

Mi propuesta es comenzar por estas decisiones y por el modelo de la actividad 1. Dejaremos preparada la organización para evolucionar hacia la actividad 2, implementando sus funcionalidades cuando corresponda.